

# TP 1

Dans ce premier TP, nous allons travailler sur le chapitre de logique vu en cours et en TD.

Le but de la plupart des TP sera de faire une partie de programmation en lien avec le chapitre travaillé et de résoudre des exercices. Les exercices devront être résolus du point de vue mathématique, avant de passer à la vérification à l'aide de ce que vous aurez implémenté.

L'objectif de ce TP sera de construire, pas à pas, des tables de vérité afin de vérifier que les tables de vérité faites dans les exercices sont correctes.

## Exercice 1

Avant de programmer les connecteurs logiques, il est indispensable d'initialiser la table de vérité en prévoyant toutes les combinaisons possibles en fonction du nombre de variables.

Déterminer une fonction qui, séquentiellement, initialise la table de vérité en fonction du nombre de variables et des noms des variables. Voici un exemple d'appel et de comportement de cette fonction :

```
> tableDeVerite <- initialisationTableDeVerite(0, list('a','b','c'))
[1] "erreur de saisie"
> tableDeVerite
[1] NA
> tableDeVerite <- initialisationTableDeVerite(5, list('a','b','c'))
[1] "erreur de saisie"
> tableDeVerite
[1] NA
> tableDeVerite <- initialisationTableDeVerite(3, list('a','b','c'))
> tableDeVerite
      a      b      c
1 FALSE FALSE FALSE
2 FALSE FALSE  TRUE
3 FALSE  TRUE FALSE
4 FALSE  TRUE  TRUE
5  TRUE FALSE FALSE
6  TRUE FALSE  TRUE
7  TRUE  TRUE FALSE
8  TRUE  TRUE  TRUE
```

Pour cela, vous devrez donc utiliser un data frame. Ce sera la base de la création de la table de vérité et, en fonction des connecteurs logiques utilisés par la suite, vous continuerez de remplir ce data frame.

## Exercice 2

Ecrire maintenant une fonction qui va permettre l'ajout d'une colonne dans la table de vérité et, par la même occasion, de nommer la colonne.

Voici un exemple d'appel de la fonction :

```
> testAjoutTableDeVerite <- initialisationTableDeVerite(2, list('P', 'Q'))
> unVecteur <- logical(nrow(testAjoutTableDeVerite))
> testAjoutTableDeVerite <- ajouterColonneDansTableDeVerite(
```

```
testAjoutTableDeVerite(unVecteur, "F")
> testAjoutTableDeVerite
      P      Q      F
1 FALSE FALSE FALSE
2 FALSE  TRUE  FALSE
3  TRUE FALSE  FALSE
4  TRUE  TRUE  FALSE
```

### Exercice 3

Maintenant que la table de vérité est initialisée, construire, à l'aide de fonctions, les connecteurs logiques « de base » : négation, conjonction et disjonction. Cela signifie que vous devez reprogrammer les fonctions : !, & et | qui existent déjà mais qui fonctionnent sous forme vectorielle. Ici, nous voulons que cela fonctionne coordonnées par coordonnées, ce que nous appelons de « façon séquentielle ». Une fois cela fait, vous vérifierez si vos fonctions donnent les bons résultats en les comparant avec les fonctions R correspondantes.

Exemple d'appel et de vérification.

```
> unVecteur <- connecteurLogiqueET(tableDeVerite[,1], tableDeVerite[,2])
> all.equal(unVecteur, tableDeVerite[,1] & tableDeVerite[,2])
[1] TRUE
```

### Exercice 4

A l'aide de vos connecteurs logiques « de base », construisez maintenant les connecteurs logiques « avancés » : disjonction exclusive, implication, équivalence.

### Exercice 5

En utilisant les tables de vérité, démontrer que :

1.  $(P \Rightarrow Q) \wedge (Q \Rightarrow R) \Rightarrow (P \Rightarrow R)$
2.  $\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$
3.  $\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$

Le travail se fait en premier lieu à la main avant de vérifier avec les fonctions implémentées.

### Exercice 6

Evaluer les formules suivantes en utilisant les tables de vérité. Indiquez, parmi les formules celles qui sont valides, satisfiables ou insatisfiables :

1.  $(P \Rightarrow Q) \vee (Q \Rightarrow P)$
2.  $(P \Leftrightarrow Q) \vee (P \Leftrightarrow \neg Q)$

### Exercice 7

A l'aide des tables de vérité, dites si les formules suivantes sont des tautologies :

1.  $P \vee \neg P$
2.  $\neg(P \wedge \neg P)$
3.  $(P \vee Q) \Rightarrow (Q \vee P)$
4.  $P \Rightarrow (Q \Rightarrow P)$
5.  $\neg P \Rightarrow (P \Rightarrow Q)$
6.  $(\neg P \Rightarrow P) \Rightarrow P$
7.  $((\neg P \Rightarrow Q) \wedge (\neg P \Rightarrow \neg Q)) \Rightarrow P$
8.  $((P \Rightarrow Q) \wedge (Q \Rightarrow R)) \Rightarrow (P \Rightarrow R)$